

Day 121: HTTP Fundamentals

1. Day number

Day 121

2. Topic name

HTTP, Client, Server, Request, and Response

Today you will learn the basic communication model behind websites and APIs.

The main ideas are:

- **Client**
 - **Server**
 - **HTTP**
 - **URL**
 - **Request**
 - **Response**
 - **Request method**
 - **Response body**
-

3. Connection

Near the end of your previous plan, you created a very small **FastAPI endpoint**.

You already saw that something could call an API endpoint and receive data.

Today, instead of writing FastAPI code, you will understand what is happening underneath:

```
Client sends a request
      ↓
Server processes it
      ↓
Server sends a response
```

This HTTP request/response idea is the foundation of most web APIs.

4. Important topics

Client

A **client** is something that asks a server for information or asks it to perform an action.

Examples of clients:

```
Browser
Mobile app
```

```
Python program
Frontend website
API testing tool
```

For example, when you visit a website in Chrome:

```
Chrome = client
```

The browser asks a server for some resource.

Server

A **server** is a computer or program that receives requests and sends responses.

For example:

```
Client asks:

"Give me product 10."

Server responds:

{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
```

The server usually contains the application logic that decides what response should be returned.

HTTP

HTTP stands for:

```
HyperText Transfer Protocol
```

Don't worry about the full name too much yet.

Think of HTTP as a **set of rules for communication between clients and servers**.

For example:

```
Client:
I want this resource.

Server:
Here is the result.
```

HTTP defines how those requests and responses are structured.

URL

A **URL** tells the client where a resource can be found.

Example:

```
https://example.com/products/10
```

You can think of it as an address.

Very simply:

```
https://
```

means the communication protocol.

```
example.com
```

identifies the server/domain.

```
/products/10
```

identifies a particular resource or path.

So:

```
https://example.com/products/10
```

roughly means:

Contact this server and ask for the resource at `/products/10`.

Request

A **request** is a message sent by the client to the server.

For example:

```
Client → Server
```

```
GET /products/10
```

The client is asking:

```
"Please give me product 10."
```

A request can contain several pieces of information, but for now focus on:

```
Request method
URL
```

Request method

The **request method** describes what kind of action the client wants to perform.

One of the most common methods is:

```
GET
```

GET usually means:

```
"Give me some information."
```

Example:

```
GET https://example.com/products/10
```

Meaning:

```
"Give me the information for product 10."
```

You will learn other HTTP methods later.

For today, understanding GET is enough.

Response

A **response** is the message the server sends back to the client.

Example:

```
Client
  ↓
GET /products/10

Server
  ↓
Response
```

The response could contain:

```
{
  "id": 10,
  "name": "Keyboard",
```

```
"price": 1200
}
```

Response body

The **response body** contains the actual data returned by the server.

For example:

```
{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
```

This JSON data is the response body.

So conceptually:

```
Response
├── Body
│   └── Product information
```

5. Foundational notes

Here is the simplest mental model to remember.

Client asks

```
"Give me something."
```

Server receives the request

```
"What resource does the client want?"
```

Server prepares the result

```
"Here is the requested data."
```

Server responds

```
Client receives the data.
```

HTTP communication normally follows a **request** → **response** pattern.

The client normally starts the conversation.

```
Client → Request → Server
Client ← Response ← Server
```

A useful analogy is a restaurant.

```
Customer = Client

Waiter's/order system = HTTP communication

Order = Request

Kitchen/restaurant = Server

Prepared food = Response
```

The analogy is not technically perfect, but it helps you remember the basic roles.

6. Basic HTTP flow

The flow you should remember is:

```
Client
  ↓ request
Server
  ↓ response
Client
```

Another way to visualize it:

```
      HTTP Request
Client -----> Server

      HTTP Response
Client <----- Server
```

Suppose the client wants product 10:

```
Client
|
| GET /products/10
↓
Server
|
| Find product 10
|
| Send product information
```

↓
Client

7. Easy example: browser requesting product data

Imagine that an online shop has this URL:

```
https://shop.example.com/products/10
```

You enter this URL in your browser.

Your browser acts as the **client**.

Conceptually, the browser sends:

```
GET https://shop.example.com/products/10
```

The server receives the request.

It may look for product 10.

Suppose product 10 is:

```
Keyboard  
Price: 1200
```

The server could respond with:

```
{  
  "id": 10,  
  "name": "Keyboard",  
  "price": 1200  
}
```

Let's identify everything:

Part	Example
Client	Browser
Server	shop.example.com server
HTTP	Communication rules
URL	https://shop.example.com/products/10
Request method	GET
Request	Ask for product 10
Response	Server's reply
Response body	Product JSON data

The whole conversation is approximately:

```
Browser
  |
  | GET https://shop.example.com/products/10
  ↓
Server
  |
  | Finds product 10
  ↓
{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
  |
  ↓
Browser
```

8. Problem statement

Suppose you have this API URL:

```
https://api.example.com/products/25
```

A browser visits this URL using a GET request.

The server returns:

```
{
  "id": 25,
  "name": "Mouse",
  "price": 500
}
```

Identify:

1. What is the client?
2. What is the server?
3. What is the URL?
4. What is the request?
5. What is the request method?
6. What is the response?

7. What is the response body?

Do not write Python code.

Focus only on understanding the HTTP communication.

9. Concepts used

For this exercise you need:

```
Client
Server
HTTP
URL
Request
Response
GET
Response body
```

The main relationship is:

```
Client
  ↓
Request
  ↓
Server
  ↓
Response
  ↓
Client
```

10. Thought process

When you see an API interaction, ask these questions in order.

Step 1: Who started the communication?

Usually:

```
Browser
Mobile app
Frontend
Python program
```

That is likely the **client**.

Step 2: Where is the request going?

Look at the URL:

```
https://api.example.com/products/25
```

The application receiving that request is the **server**.

Step 3: What action is requested?

Look at the HTTP method.

Example:

```
GET
```

Think:

```
GET → retrieve information
```

Step 4: What resource is requested?

Look at the URL/path.

```
/products/25
```

The client wants information associated with product 25.

Step 5: What came back?

That is the **response**.

If the server returns JSON:

```
{  
  "id": 25,  
  "name": "Mouse",  
  "price": 500  
}
```

that JSON is the **response body**.

11. Beginner-friendly pseudocode-style request flow

You are not writing real code today.

Think of the process like this:

```
START  
  
client chooses a URL
```

```
client creates a GET request
client sends request to server
server receives request
server checks which resource was requested
server prepares product data
server creates response
server sends response to client
client receives response
client reads response body
END
```

Or even shorter:

```
CLIENT:
    request product 25

SERVER:
    receive request
    find product 25
    prepare response
    send response

CLIENT:
    receive product data
```

12. Suggested solving approach

Use a **conceptual API approach**.

When given something like:

```
GET https://api.example.com/products/25
```

break it into pieces.

```
Who sent it?
  ↓
Client
```

Where was it sent?
↓
Server

What type of request?
↓
GET

What was requested?
↓
Product 25

What came back?
↓
Response

What actual data came back?
↓
Response body

A useful mental template is:

```
[Client]
  |
  | [Method] [URL]
  ↓
[Server]
  |
  | process request
  ↓
[Response body]
  |
  ↓
[Client]
```

13. Easy edge cases

Edge case 1: Server unavailable

Imagine the client sends:

```
GET https://api.example.com/products/25
```

but the server is offline.

The expected product data may not be returned.

Conceptually:

```
Client
  |
  | request
  ↓
Server unavailable
  |
  X
No normal product response
```

Possible reasons include:

```
Server is down
Network problem
Temporary service problem
```

For now, just understand that a request does not always successfully reach a working server.

Edge case 2: Wrong URL

Suppose the correct URL is:

```
https://api.example.com/products/25
```

but the client requests:

```
https://api.example.com/prodcts/25
```

Notice the typo:

```
products
prodcts
```

The server may not know what resource the client is asking for.

Conceptually:

```
Client
  |
  | wrong URL
  ↓
Server
  |
  | resource/path not found
  ↓
Error response
```

A URL must point to something the server understands.

14. Expected request/response explanation

For this example:

```
GET https://api.example.com/products/25
```

with the returned data:

```
{
  "id": 25,
  "name": "Mouse",
  "price": 500
}
```

you should eventually be able to explain it approximately like this:

A client sends an HTTP GET request to a server using a URL.

The URL identifies the product resource the client wants.

The server receives the request and processes it.

The server sends an HTTP response back to the client.

The product JSON is contained inside the response body.

The most important picture to remember is:

```
          REQUEST
Client -----> Server

          RESPONSE
Client <----- Server
```

15. Hint only

For the exercise, start with the direction of communication:

```
Who sends the GET request?
  ↓
That is the client.

Who receives it?
  ↓
That is the server.

What describes the location?
  ↓
```

The URL.

What comes back?



The response.

Where is the actual product data?



Look inside the response.

Day 121 key takeaway

You do not need to memorize complex networking details yet. Remember this simple foundation:

HTTP lets clients and servers communicate.

Client sends request



Server processes request



Server sends response



Client receives response

This mental model will make later topics such as **HTTP methods, status codes, headers, JSON APIs, REST APIs, and FastAPI endpoints** much easier to understand.